Module 17 Transcript

Video 1: Module introduction

In deep learning, a convolutional neural network or CNN is a class of deep learning architecture commonly applied to analyse computer images in what's known as computer vision. Nowadays, they have applications in other domains and have been extended to other types of data too. For example, CNNs have successfully been applied to natural language processing and to financial time series.

CNNs, as opposed to fully connected networks, take advantage of both spatial correlation between pixel intensities and computer images and of the hierarchical pattern in data. The network assembles patterns of increasing complexity using smaller and simpler patterns in bus and their filters. Consequently, CNNs tend to use substantially fewer neurons in their architecture, making them less branch overfitting than their fully connected counterparts.

An important mathematical concept in CNNs is that of convolution filters, a term computer scientists borrowed from their electrical engineering colleagues. We will describe the anatomy of convolution filters and how they work. We will then move on to describe how convolutions are using computer vision applications and which types of tasks enterprise-grade CNNs can perform.

I hope this is evidence enough for the far-reaching usefulness of CNNs. Let's get started.

Video 2: Why convolution matters

To grasp the motivation for understanding convolutional networks, consider the following example. Imagine you have an image classification problem where you're trying to teach in your network how to classify cat images. Let's say you start using some high-resolution images with 1000 by 1000 pixels, which are coloured, and which have three coloured channels. For such a large image, the input layer of your network would require about three million input features, one for each pixel. And if your middle layer had, say, 1000 units, the network would have three billion parameters to be determined. With these many parameters, it's very difficult to get enough data to prevent overfitting.

The relative size between the number of weights and the size of the data set could imply that our network will likely overfit the training data. Notice also that the training time and the memory requirements with the millions, if not billions, of parameters could be a bit unfeasible. However, for computer vision, you don't want to be stuck using only tiny low- resolution images because your network's performance would fall short and perhaps confuse puppies and cats and etcetera. This is where convolutions and the weight-sharing property of convolutional neural networks come into the rescue. In the next presentation, we will illustrate the concept of convolution filters with various examples.

Video 3: Convolutions in computer vision

Does this image look familiar to you? It's a famous painting from Caravaggio called The Card Sharks. This is not going to be an art lecture, but we're going to use it to illustrate the concept of a convolution filter. Let's see how different convolution transformations would modify this image.

To begin, and to simplify this visualisation exercise, I'll replace the original image with the grayscale version. Here's what happens when I apply horizontal edge filter. Notice the appearance of highlighted vertical lines, since the filter is operating from left to right and therefore notices vertical contours. When we apply the convolution filter, we're essentially creating a new representation of the image, highlighting important features of the original image. That would be helpful in some computer vision tasks, such as image recognition or object recognition.

Let's see what a vertical edge detector looks like. As you can see, now the effect is that of highlighting the horizontal or near-horizontal contours. Blurring filters are another common type of filter. These are widely used to anonymise important data. They work by averaging small patches of the image.

And finally, max_pool type of filters take a patch of pixels and replace it by the max value of all pixels there. This gives the image a vintage video game feeling, like those old Atari games. In computer vision, max_pooling is sometimes useful to reduce the size of the images while at the same time to retain some relevant features. For this particular example, my original grayscale image had 387 by 620 pixels versus 48 by 77 in the max_pooled version.

One caveat, though, is that in CNNs, one doesn't specify the convolution filters when designing the network. But instead, let the convolution network learn the features that it needs to classify images well.

Video 4: Why CNNs?

Suppose that the first two layers were fully connected, and recall that for fully connected layers, the number of weights is the number of input dimensions times the number of output dimensions. In this example, that corresponds to about 3072 times 4704, which is approximately 14,000,000 parameters to be determined. Using the parameter sharing property of CNNs, which means we're using the same set of weights in different parts of the image, we can reduce that number down to less than 500 parameters. We're reducing the number of parameters by almost four digits. That's a massive economy in terms of computational time and memory requirements. Less weight parameters means that we're less prone to overfitting and we can learn more with less data.

Video 5: The biological basis for convolutional networks

According to the experiments conducted by Hubel and Wiesel in the '50s, the visual cortex has small regions of cells that are sensitive to specific regions in the visual field. This means that specific areas of the visual cortex will be activated when

specific regions of the visual field are excited. The activated neurons also depend on the shape and orientation of the objects in the visual field. For example, vertical edges cause some neurons to be excited. And horizontal edges will cause some other neurons to be excited. The neurons are connected using a layered architecture. And this discovery led to the conjecture that mammals use these different layers to construct portions of images at different levels of abstractions.

From a machine learning point of view, this principle is similar to that of hierarchical feature extraction, similar to how neural networks operate. Though there were early attempts in computation in neuroscience, the leapfrog design that led to convolutional neural networks was due to LeCun, who, in the '90s, created a weight-sharing mechanism between layers, making CNNs especially convenient for image recognition tasks.

Video 6: Dissecting the LeNet-5

We're going to focus here on CNN architectures using image recognition tasks. That means that the prediction task is: what is this image? So, what your network needs to learn is a series of convolution filters that make it easier for it to classify or detect the class objects you're interested in. For example, is this a car, or is there a pedestrian in this image? CNNs normally consist of three blocks: convolutional layers, pooling layers and fully connected layers. We normally refer to them as conv, pool and FC. The typical architecture of a CNN consists of a stack of alternate conv and pool layers, and towards the end, we added FC layer or two.

One of the first examples of CNNs was the LeNet-5, designed by Yann LeCun in the '80s for the digital recognition task. Let's enunciate each component for the LeNet-5. The input consists of the image of a digit, 32 by 32 pixels, with three colour channels called RGB channels or red, green and blue. So, we say that the image is represented by a 32 by 32 by three tensor. After the first convolution, our original image transforms into a 28 by 28 by six image. This is a consequence of applying six filters of dimension five by five by three with a stride of one along the height and width of the original image.

Notice how the filter has the same depth as the original image. That's a conventional image processing. Also, by convention, we'll call the resulting 28 by 28 by six tensor a representation of the original image. It may not resemble an image recognisable by a human, but it does contain most of the relevant information necessary to recognise the original image. Then it goes through a max_pool layer, which reduces the size of the previous representation. Then again, it goes through another convolution layer and another max_pooling layer.

And finally, towards the end, we flatten the tensor representation and push it through two fully connected layers until we reach the output. A ten-dimensional vector containing the network guesses, a probability vector. For what digits could the original image correspond to? For example, it could output that it's 25 per cent A1 and a 75 per cent A7. So, that's the gist of CNNs. In general, you learn a lot by inspecting the source code and research papers of famous CNN architectures such

as ResNet, AlexNet, VGG, GoogleNet for example. That's it for now. There's lots of design trends that go into creating a new CNN, but my word of advice is the following.

Check the open-source code and see if there is any existing architecture that solves a task similar to the task that you have at hand. For example, if it's image recognition, object detection, face recognition, AI art, etc. You're likely to have to tweak the network for your own purposes, but you don't need to face a code start. Also, notice that in many CNNs the trend is to use less layers and better hyper-parameter optimisation. As you go deeper into CNNs, the representations are getting smaller and deeper.

That means that the height and width of your representations will shrink, but the depth of the tensor representations will increase. On the other hand, as you go deeper, your representations become information-rich and provide more abstract representations of the following data set, as proposed by Honglak Lee, and using a data set consisting of face images. We spend the rest of this session understanding how to compute the dimension of inputs and output representations at different layers in a convolutional network and use that information to count the number of weight parameters to be determined in a network.

Video 7: A convolutional network in PyTorch

In this demo, I'd like to show you how we train a convolutional neural network in PyTorch by replicating the LeNet-5 network and by training it on the CIFAR-10 data set to identify ten different classes of objects, including cars and cats.

The CIFAR-10 data set is a collection of some 60,000 thumbnail-sized images in colour. It's a sample of an even larger data set consisting of 60 million images captured by a web crawler. The set was curated by Alex Krizhevsky from the University of Toronto. This was one of the first large-scale experiments on unsupervised learning using convolutional networks. Our goal here is to cover some of the image processing built into PyTorch, which we will use for loading and processing our images. Then we will show how to define a convolution filter in PyTorch. And finally, we demonstrate how to train and evaluate our network. Let's get started.

First, we download the data sets from Krizhevsky's web page. The raw files are in the compressed format, but the PyTorch API unpacks and loads them in batches for you, as specified by the batch size parameter. Here's the data loader. Notice that you can also specify pre-processing functions to manipulate the raw images, as you can see here. In our case, we are normalising the pixels' values in each channel of the image to have mean 0.5 and standard deviation of 0.52. This has to do with the nature of the RGB scale, which ranges from zero to one.

In the PyTorch documentation, you can verify that for the API for the normalised function, the first input corresponds to the means, and the second input corresponds to the standard deviations. Since each image has three channels, we have to specify a sequence with three numbers. Finally, for convenience, I'm also

defining here a sequence of classes to specify the class name of each object in our collection. So, we have ten classes here, starting with planes and ending with trucks. I'm also defining here a sequence vector to function as a dictionary between labels and objects for future reference throughout the demo. Now, let's scroll down to the definition of the network.

I'm replicating here the LeNet-5 model. Since we've already seen linear layers in previous examples and also rectifier layers, I'm going to focus here primarily on the convolution and max_pool layers of this network. To specify a convolution layer, you need to specify the filter size here, the last number and the number of channels of the input and the output. For the max_pool you have to specify the filter size, which is the first parameter here, also two, and the size of the strides. So, for this specific example, I have a filter of size two and stride of size two. The max_pool filter acts the same way in each channel. So, the max_pool filter doesn't need to be fed the number of channels of the input. Everything else in this network architecture should look familiar to you already. The print function in the Jupyter notebook will print in detail what goes into each parameter key, in case you get lost during the design. The remaining steps are quite simple.

We have to define or specify a loss function. In this case, we've chosen the cross entropy. And we need to specify an optimisation algorithm. So, here I've chosen the stochastic gradient descent with momentum. I decided to switch on the momentum to slow down a little bit the zig-zagging that the stochastic gradient descent goes through. Next, we need to set the main learning loop where the learning takes place. For this particular example, we'll loop through the data set twice. So, this is the number of epochs. And the steps that we're going to follow here are pretty standard. We batch some data. We zero out our gradients. We do a forward pass, which you can see here in this line. Next, we evaluate the loss function for the batch that we were given. And then we go backward using backpropagation to compute the derivatives. The last step is simply to call the optimisation method to update the weights for you. I've also included here a helper printing function that prints for me how the accuracy of my network is improving every 2,000 batches or so. Let's scroll down a little bit to see how our network is performing.

I'm going to sample here four data points from our test set and see how well our network is performing on that batch. You can see here that for my batch of size four I got four pictures-- a cat, a ship, a ship and a plane. If I actually call my network and ask it to predict what the test set would look like for that specific batch, it gives me the following answer. So, it gets two of the answers correct and misses the other two. If you want to see how the network is actually performing on the entire data set, I've included here some help functions that will show you what is the accuracy of the network for each one of the classes in the data set. The network seems to be doing pretty well, much better than random, since a random guesser in this case would only get it correctly ten per cent of the time. The network seems to be struggling, though, with cats and deers. Can you guess why?

Video 8: Guest interview with Andrea Dunbar

Chris: I'm here today to talk with Andrea Dunbar and really a pleasure to have you here today. She's the group leader of Edge Artificial Intelligence and Vision at CSCM was based in Switzerland, originally set up to do research and development for the watch industry and now expanding into a whole bunch of different areas. Really interesting project. So, welcome Andrea.

Andrea: Thanks for inviting me today here Chris.

Chris: It's great to have you. I'm wondering if we could start by just you could tell us a little bit about your career path, what you've been doing over the years and then maybe pivot a little bit and talk what got you into artificial intelligence, machine learning, etc.

Andrea: Sure. Yeah. So, by training, I'm a physicist or I did an undergraduate in physics at the University of St. Andrews. Actually, after I finished that I was a little bit annoyed with physics quite frankly and I went off into the city of London to do some brokering and to join the financial world. And actually, quite luckily, that told me about how much I truly did love physics and that actually then decided to go back because I missed it so much and I went and did a PhD at Trinity college in Dublin.

And it was around optics and after having finished my PhD I moved to Switzerland to the EPFL, which is where I really started getting interested more in vision systems and computing as well, but still predominantly in vision and then moved to CSCM two years after my postdoc started at EPFL. And there was when I really jumped from doing optics primarily. And quantum electronics and optics and moved the modeling that I had done more towards a purely data science elements. So, as you can see the name Edge AI and Vision is that we still do vision systems but more and more we work on artificial intelligence. We also work on what I would call standard signal processing and though I'm a big fan of artificial intelligence because it allows you to do many things that standard signal processing can't do. I also like physics and standard signal processing. So, I should be honest about that. It's not the panacea artificial intelligence and there are many standard techniques that also work very well.

Chris: That's really interesting. I'm just wondering if you could take us a little bit into what edge computing is and what it means.

Andrea: Sure. So, I mean this edge versus cloud debate, it's a very good question Chris because edge computing is… can mean a lot of different things to different people. So, I think it's very important to make it very clear what we mean when we talk about edge computing. And edge computing for us means when you're making the the algorithms or you're processing the data close to the sensor, so where you are sensing the data as opposed to cloud computing, where you take the data and you send it up into some central server. So, edge computing in itself means working the data near where the sensor is and then you can have how close to the sensor is. But for us it's really at the edge. So, it's when the processing unit of the data is

next to where you're sensing.

Chris: I see. So, then this would be basically this refers to doing the processing of sensor data close to the place. So, in a device close to the basically on the device or near the device I should say.

Andrea: So, that's again so it can be a little bit subtle. So, you can have a device which is really a camera and behind it a CPU so really a processing unit which is integral to the camera. And then you're actually working on the edge there. But you can also for example, as we are now have it on a on a computer and then you're also processing on the computer which is right next to the sensor. And sometimes people distinguish between very edge, which is where you're really actually processing in the same unit as the camera and then edge where you may be processing it on a computer next to the sensor. So, that's also a subtle distinction between the two.

Chris: Nice. Okay, I wanted to talk to you about convolutional neural networks actually and what their role is in edge computing. Why is it important or how is it been changing or developing over the years to get some of your feelings about the trends?

Andrea: Yeah. I mean the if I start with the trends and edge computing to explain a little bit why it's taken off because I think before there was this big element of why the cloud is so amazing and all this computational power that was available. But edge computing has certain advantages and which are basically that you don't suffer from latency. So, that means the speed of what happens at the edge can almost be instantaneous. So, you can really treat your data very quickly. And also you you don't need to be connected to the cloud. So, sometimes we call something like critical connectivity that is for example, if you're monitoring a driver who if they're tired or not, you want to be able to do it without worrying about your connection to the cloud and you need to be, you need to be able to do it in what we call real-time, it's no good saying, "Oh yes, he was, we saw he was tired five minutes before the accident but you know later on, you need to know it in real-time." So, this this real-time aspect, this low latency and that often comes in also with it comes in when you have many fast automated processes such an industry, like if you're looking at defect detection or something which can go at tens of meters a second papers on a roll then you really need to be able to very quickly see if there's a defect and stop.

So, I think speed, critical connectivity and then there's also the fact that more and more we're wanting to monitor many, many different things. So, to monitor many different things you have to keep sometimes low power so you don't want everything plugged in. So, something else that's coming in on the edge at the moment today is what we call always on devices. These are devices where you don't necessarily need to power supply where there's full autonomy in the device that can run off ambient light through a photo voltaic cell or you could be a long life battery where you could place it maybe for ten years. And then and so these need to be very ultra low power, low, low energy devices. And then finally, I would say

there were, there's also the fact that people are trying more and more to hold their privacy locally so they want their data treated locally so that you can turn just the metadata into the cloud. That is to say if I want to know the number of people in a room, then I don't need to know who they are. And by treating the data locally, I can extract the number of people and just send that metadata which is the number of people in the position and all the time, keeping those people's privacy intact. And so this is one area as well why edge computing is very important.

Chris: That's really interesting. And then we talk a little bit about convolutional neural networks and how they fit into this picture would be.

Andrea: Yeah. Exactly. So, convolutional neural networks have really taken off really with the one set of deep neural networks and the work by Yann LeCun in the 90s. The reason a convolutional neural networks are so powerful today, why they're so used, I think principally are for two reasons. The main one is they work their extreme. It sounds strange to see but they're extremely accurate and accuracy is everything when you're looking at extracting information from an image or from data, some form of data. So, if you have something that that works very well.

And the question is, why do they work so well? And I will come back to that, but maybe before I go into why they work well, I just would like to say the second reason is they're very easy to implement. So, these the fact is is that you don't need years of experience to set up a convolutional neural network, you can do it and there's little pre-data processing, you will hear a lot about data scientists who say, "Oh, I'm labeling the data and processing the data, I'm organising the data and setting up the data pipeline." But actually CNNs are very light compared to a lot of alternatives in terms of data processing. So, the accuracy and the ease of use of the main things and just to come back on the accuracy and the way that convolutional neural networks actually work. And we often talk about the fact that they work very similarly, at least initially to the animal way that our visual system works. And I think that this we are very efficient in terms of all the processing that we do. And this convolutional neural network is also very efficient and what it does, the way a convolutional neural network works is to pick out features on really a very fine scale, but in a very generic way.

So, you would start by if I give an example that's, is there a bicycle? Is there a bicycle in this image? So, what are the key traits of a bicycle? Well, you're going to have some straight lines, where the where the bars are, going horizontally or at an angle, you're going to have small bits of curved because there's normally going to be a couple of wheels involved in a bicycle and it's going to pick out these features and it's going to pick out these what we call very low level generic features which can be also in other images. But then it's going to go over and it's going to make a hierarchy of these features. So, first it's going to look for these and then it's going to look for more and more complex features. And this hierarchy of features that it has is sort of automated in a convolutional neural network. We don't have to decide how that hierarchy works.

It will choose the features but it will also choose the hierarchy and this is a huge

saving for the computer scientist. Before, what we would need to do is we would need to choose the feature, work out the algorithms to get that feature and then we would need to structure that feature in a hierarchy, but now we don't need to do that. A convolutional neural network will take that and it will work out the hierarchy of the features as well as the features themselves. And this energy saving, this ease of use means that you can really actually train them very well and they're very scalable, very simple to use as a neural network. And I think this is one of the powers of convolutional neural networks.

Chris: I think maybe you could just amplify a little bit on the idea of brain inspired neural networks or the analogy to that.

Andrea: Absolutely. Because, in fact the first layers of a human cortex system or in certain systems is that when we have an optical system and it works very similar to a camera, you have an optic and then it will go on to a part that is a photo receptor.

So, an image in a camera, but for us it's a retina and in the retina you have rods and cones. And these rods and cones are already designed with what we call some pre filters which are looking for some very specific features which help us very quickly work out what's of interest in a scene. And these sort of techniques, so this one is, this feature extraction is very much linked to a convolutional neural network. There are other ones as well which are not based on convolutional neural networks but are also things that have been integrated into our vision systems which are differences. So, we're often looking for differences, if nothing changes within an image then it's not interesting and what's of interest is normally what changes, not what's static. And so this is also a type of feature that we have in the biological system and in the computing systems we put into place. And well, I am personally a big believer in looking further and further into biological systems which have been adapted for… and this is particularly interesting, because it's adapted for human understanding. So, if you think about a convolutional neural network, it's working similarly to the way that we're working and we're also just like computers. We're getting huge amounts of data but little of it is relevant, a little bit is necessary and the convolutional neural network and this feature extraction that you have is one way to basically sift, compress the data to only extract the relevant element of the data.

And this is also what we have to do because the visual cortex, the part of the brain, it's actually the most power hungry part of the brain system. It's extremely computational intensive, even for the brain, to actually to work out what's in an image and therefore this extraction, this efficient extraction is extremely important. And it's also what allows us to have such a complicated visual system and use it.

Chris: Maybe we could talk a little bit about now the edge, go back to the edge computing case and think about an example of a device and maybe how it would work with the CNN. Is it that they would be built onto a chip and put in with the processing units? How would it work? And what would be an example of that?

Andrea: Okay, so I give you a quite a nice example, I hope it's okay to say that we've developed in at CSEM recently, which involves an always on device. So, the

idea is we want to do people recognition but we want to do it all of the time. We want it to be working in less than a milliwatts. So, really something that's completely autonomous, you don't need to plug it in, you don't need to worry about it, you can just put it there with a battery and you've got years of lifetime. And in view of that, what we actually did is we set up a hierarchical intelligence system. So, the first part would be a very simple, like a binary decision tree. Just is there a person in this image, so there you don't need to know who it is, you just really want to extract very basic features.

Yes-no sort of algorithm can work. And this can work at really just hundreds of micro watts. So, it's very, very low power. And then only when you need to extract the key features, would you say, yes, this person is looking at this at this imager and yes, now we want to wake up the convolutional neural network and the convolutional neural network will be used to actually extract the features. And just to be concrete and what it will do, you can actually look at the eyes, look at the distance between the eyes, the shapes of the eyes, identify the center points of the eyes and then measure these distances by flattening the face in the field. And this can then allow you to know whether this is the same Chris Tucci now or whether it's Andrea Dunbar or someone else. So, and I think that this is a nice example.

It's actually on a chip now. So, this is actually done in a silicon chip and it was designed to do this and that's a general trend in edge computing. At the moment, we have many players on making chips for these very specific applications or trying to make the generic chip that will work for all applications, which hasn't yet happened. But this is actually where edge computing is going and this is we see the potential but we don't yet fully have the winning chip, the amount of memory, the amount of processing power that's needed. And what we do see is that we're making multi-core chips which are specifically helpful for edge computing with convolutional neural networks, like GPUs are useful for CNNs because they allow parallel processing of the data and this is very important.

And we're moving beyond what we call the von Neumann configuration where you have in any process or a part of memory, a part of processing and then you're feeding in linearly, from the memory you're making the multiplications. This is very inefficient for convolutional neural networks and generally speaking, deep learning. So, what we're seeing is these multi-core chips which are coming onto the market and there's many very nice, very rich startups working on this.

Chris: Great. Well, I think this is a really nice overview and also this last example of the image detection and recognition is super helpful. I'm sure that the participants in this programme are going to get a lot out of it. So, thank you very much for taking the time today to talk with us and give us your insights on how things are changing in edge computing, specifically with regard to CNN. So, thanks again.

Andrea: Thank you very much, Chris.

Video 9: Guest interview with Neil Rubens

Chris: I'm here today to talk with Neil Rubens. And Neil thank you very much for coming today to talk a little bit about artificial intelligence and machine learning.

Neil: Thank you. It's my pleasure.

Chris: Maybe we could start with you explaining a little bit about your career path, what you've been doing, what you studied, where you've worked and what got you interested in artificial intelligence and machine learning.

Neil: I see. So, I guess it's going to be a long story so.

Chris: Take your time.

Neil: Later, but I guess, I become interested in AI by playing games. You always wonder what the computer is doing and why you cannot with it or you try to predict what it does. And so that was against the initial interest. And then, I studied Computer Science. And I was actually lucky to get AI related jobs right away. So, I was a research assistant on Nissan, project business on that was doing actually adaptive cruise control that was like 20 years or so.

And now I actually have.

Nissan is that device probably not programmed by me because it works really well. But so that was actually really exciting to be able to see, things actually working. Then, I worked at Coral and worked perfect. And there too I was lucky, I got to work on it was called perfect assistant.

So, it's the perfect version of Clippy from Microsoft Office. And probably some people are much younger than me. So, in case I didn't know that word perfect used to have 90 per cent share of office software. So, that was, the heydays unfortunately didn't last too long for me. Afterwards, I worked as a Software Engineer too for company called Utah or NIC, National Information Consortium. So, basically they were doing data processing for different states. So, I was in the state of Utah. So, we've analysed the mv data, e-commerce data, etc. Then, I worked for a subsidiary of Bell doing the business telecom analysis. Then, I really wanted to go back to graduate school. Like, I wanted to go to graduate school earlier but I had to make a living. So, I went for masters. I said my masters I did on Information Retrieval. Because dealing with text, I was always wondering how things worked. So, I did that. And then, information retrieval is a little bit of NLP and and machine learning. So, then I wanted to know a lot more about machine learning parts. So, I did my PhD and during PhD, I worked on model selection and active learning. And then, in the context of recommended systems. So, I was pretty, I would say hardcore engineer or machine learning. And then, through happenstance, I met my colleague Marcia Russell, Dr. Marcia Russell at media at Stanford. And then, started to work with her and so what media X does is, there are a lot of big corporations that come to Stanford and they want to do research or proof of concepts but it's usually interdisciplinary or maybe an area that hasn't been defined. So, media exports RFP together brings a lot of different labs together and kick start

those projects. And so then, actually I started to do, being exposed a lot more to the business side of things. And so what we did was analysing start-ups, trying to see which areas are being popular, the dynamics… variety of aspects. And I think it was pretty successful in academia and government. So, then I decided to do a startup basically commercialising technology. So, I made very typical mistakes probably academics and engineers make, not knowing the domain. So, it turns out what business people want is very different from what is needed in academia and government. So, I didn't lose money but didn't make much. And so but I really got excited about this intersection between machine learning and business. So, now I'm actually working for Visa and they have a unit that is called Visa Consulting Analytics. Basically, the one that works with all the big banks, big merchants and trying to analyse huge volumes of Visa… that data that they have and try to improve operations or strategies, many different aspects. I'm sorry, that's a very long.

Chris: But that's really, no that's excellent. Thank you for that because I think it also gives a lot of different applications of machine learning that you've, that you've actually done and worked on in different sectors. I think that's actually really, really relevant. Let's talk a little bit about recommender systems or recommendation systems. And what you do? How do you think about them? What are they good for? Maybe give a couple examples, use cases that you've worked on. And just in general, tell us a little bit about recommender systems.

Neil: So, my view is probably biased in the positive way. So, I think recommender systems are maybe one of the most crucial things. Because for example, if you know what you want, it's fairly simple. There's a Google, you put query, you get the results. And so recommender system, it takes it to the next level is. What do you, how can you suggest something? But you don't, you yourself don't quite know what you want. And so this is actually hard because. And I think it's really important because when we get good recommendations, we are really happy about it. And company makes money, everybody is happy. And so there are typical ones like Netflix, recommending movies or Amazon, recommended products. But it's actually in my, in my view, almost everything could be framed as recommender systems problem. For example, as a company, you often services or you're giving promotions. So, those are all different recommendations that you can actually provide to the customer and usually have thousands of options or different things. And there's always limited amount of time that the user has and the company has. So, like how do you optimise enough for both parties is very important.

Chris: Yeah, that's really interesting. So, what promotion, what would be an example of a choice that you have to give or offer, decide on that might be tailored to an individual online, for example.

Neil: I see. So, I think that's one thing that also there are so many different metrics that both companies. So, it's usually there are also quite conflicting often. So, of course, a company tried to maximise profit but it could be no short-term profit, could be long-term profit. For customer too sometimes, ideally they want something cheap and good. But of course that is rare. So, then you need to decide which way to go. And then, there is a matter of many different aspects. The quality of the

product, how longevity, personal preferences, etc so. And that's the tricky part is, you have to do it very quickly. When the person goes to the website, you have to somehow figure out what they want or maybe through some interactions, may be the initial query or some initial clicks. You already tried to figure out, which products out of millions of products that you might want to show to the customer.

Chris: Actually that's really interesting because normally we think of everything as being search oriented. Because you type in something on a website. And then, you have to decide what to show the person. So, that's pretty clear. But actually you just reminded me that there's a lot more other behavioural things that you could, that you could look at besides pure search terms for recommendations.

Neil: Exactly.

Chris: Great. Let me ask you a little bit about cold start active learning. So, I wasn't familiar with this term and I think you've worked on it a lot. Maybe you could tell us a little bit about how that works, what it's used for and give us some examples.

Neil: Yeah. Thank you. So, I likewise, I think it's one of the most important issues which is surprisingly overlooked. Like I'm always puzzled. But so the cold start issue, I guess the name is pretty good. For example, you have a customer, a new customer that have never been to your site or have never used your service or your company. And the problem is that you don't have almost any information about the customer but you have to make as good recommendation as possible. Because if the customer doesn't like your recommendations or doesn't like the services that you offer the customer, they will never come back. So, you will lose them and it's, it's pretty important. So, how do you make recommendations when you don't know anything about the person. And also from business sense standpoint is also tricky because usually user acquisition is quite expensive. So, you spend a lot of money acquiring the user, you brought them to the site. And then, you really have to make a good impression right away. And it's, it's difficult.

Chris: So, how does it work exactly? How does it, what techniques do you use to to attack this problem of relatively unknown person showing up, interested in, something?

Neil: Yeah! And also one thing I should actually clarify that Cold Start is not only for people, it could be also for products. Let's say you've got a new product at your company. And you don't know which, which users like it, which users don't like it. So, this is just another example of Cold Start, is basically instances of when you don't have much data about something. And I guess how you solve it is, at least a good way is with active learning. So, what active learning does is basically try to estimate which information is useful or not or how… how informative something is. So, there's some classic examples like seeking an award and asking people to ask 20 questions. So, those 20 questions are very important because you want to basically divide as much of search space as quickly as possible. So, usually the best division is in half. So, if it's a binary then, binary search is pretty powerful because if you keep dividing in half after 20 questions, you can really get pretty, pretty, pretty accurate. And so for example some classic ways to to show it to like

for example for recommended systems, a typical mistake that we probably all see when, when you search for something, it just shows you to many of the same product. But sometimes actually you might want to show different categories because maybe the user is not sure. So, don't kind of don't narrow them down too early. So, I think in the beginning you want to do kind of breadths and then you narrow down. I think also this type of interaction is also kind of related to the problem of exploitation exploration. So, exploitation is companies try to recommend something right away to just sell to the customer, but the thing is you should take time to learn what the customer likes and let them browse around maybe intrigued them with some products categories that have never looked at but might be interested in, then you actually get to know a lot more about the customer. And I think if you do it right in the long-term you will actually, the LTV would be much better. A long, long-time value.

Chris: That's interesting. So, what you're saying is don't try and be too precise too early because you think you might know maybe you're not sure exactly what they're what they're looking for in any sense. I think we've been focusing a lot on e-commerce here.

I'm wondering if there's other kinds of examples beyond selling something online through a website or something like that. Is there some other use cases that we can imagine for this sort of thing?

Neil: So, sorry I kind of went to business.

Chris: It's good, it's good, it's excellent.

Neil: So, the classic example is usually actually coming from medicine. So, for example you have a patient who comes in, you can order in a very expensive MRI for them. So, information will be very good but it will be very expensive. And also it will be maybe not so comfortable for patients because they have to go through the procedure and costly. On the other hand, you can measure the temperature or you can get the blood test. So, the question is which… which test would you do? Like which test will provide you the best information at the lowest cost? Another one, from oil is like basically where to drill and how to how deep to drill. So, it's right, I see what you mean.

Chris: So, the analogy is that if you drill you mean you're going to go straight in and that's super expensive and you may or may not pan out, and so you're better off making some smaller tests, try and figure out exactly what would be the best place to be. And I guess even that people use machine learning even for that problem I'm guessing, guessing, very nice. And can you tell us a little bit about what you're working on now in the sense of you know working at Visa, Is there anything that you can tell us about that?

Neil: Sure. So, I think so the team I'm on is very kind of very interesting team. So, most of members are pretty senior. So, we had a lot of consultants from business side and we had a lot of data scientists. So, then we basically go to business clients and Visa has many different types of clients. So, there are banks that have Visa

debit cards and credit card, the credit card companies of course they're also merchants and so they also have different questions. So, basically, we listen to what they want and then we try to provide solution that works for them and for that of course it's usually we utilise our business data, so it's really extensive I think. Don't remember the number but I think trillions of dollars actually goes through Visa, so it's really very rich data sent.

Chris: I imagine you must have billions of transactions and in addition to probably information or some information about the people who use them in the merchants who sell things etc. etc. So, you can probably look for patterns in there and then try and help the person that you're dealing with on the on the business side then.

Neil: Also I forgot to mention also that we often work with government agencies as well, especially tourism boards. So, they want to know who came but how can they make it better for government is for like some economic indicators as well. I think the beauty of vision that data is very timely. You can get up to date data right away and large-scale data.

Chris: That's an amazing treasure trove of data start with to mine for to look for answers for different kind of problems. Great well listen, let me thank you very much for taking the time today. I really appreciate it. I think it was really useful to hear about some of these examples. Also what you've worked on has been very interesting and inspiring. So, let me thank you again for coming in today and for participating in this interview.

Neil: Thank you, it was very fun. Thank you. I think it's a great, great endeavour for businesspeople to know more about machine learning and then the other way around.

Chris: Thanks a lot.

Video 10: Module summary

In this module, we had a mouthful of information.

We've learned about how the visual cortex in animal brains inspired the design of CNNs.

What are the LeNet-5 building blocks? and how to programme and optimise our own CNNs in PyTorch? So far, we've been discussing supervised and unsupervised learning.

In the next module, we will learn about a third type of machine learning called reinforcement learning.

This means learning from trial and error as opposed to having a teacher telling you what to do or not to do in each situation.

But before we understand reinforcement learning, let's take a detour to the casino.